

Rapport de Projet — NeoBank

Sommaire

1. Contexte et objectifs
2. Architecture générale
3. API Backend (FastAPI)
4. Application Flutter
5. Base de données et migrations
6. Validation, sécurité et gestion de session
7. Tests
8. Conclusion et améliorations
9. Annexes — Captures d'écran

1. Contexte et objectifs

Objectif : réaliser une application mobile de démonstration permettant d'analyser des cryptomonnaies (historique + prédiction), de simuler des opérations bancaires en USD (solde, transferts, bénéficiaires) et de gérer un portefeuille crypto (achat/vente, positions, PnL), avec une API REST documentée via Swagger.

Fonctionnalités principales

- Authentification (inscription, connexion, changement de mot de passe)
- Marché : recherche intelligente de crypto (CoinGecko id), historique, prédiction, graphique
- Banque : solde USD, transferts, historique de transactions, gestion de bénéficiaires
- Wallet crypto : achat/vente, positions, PnL (+/- en USD et %)
- Watchlist / favoris (★) pour suivre des cryptos
- Ordres simulés : LIMIT / STOP-LOSS / TAKE-PROFIT (création, liste, annulation)

2. Architecture générale

Le projet suit une architecture client-serveur : Flutter consomme une API FastAPI. Les données applicatives sont stockées dans SQLite (wallet.db). Les données de marché proviennent de CoinGecko (recherche, prix, historique).

Composants

- Frontend : Flutter (Android / Web)
- Backend : FastAPI (Uvicorn)
- Base : SQLite (wallet.db)
- Services externes : CoinGecko

3. API Backend (FastAPI)

Démarrage

Commande : `uvicorn main:app --reload --host 127.0.0.1 --port 8000`

Documentation Swagger

La documentation interactive est disponible via `/docs`. Les endpoints sont regroupés par domaines : authentification, marché, wallet, banque, bénéficiaires, ordres.

Table des endpoints (extrait)

Domaine	Endpoint	Auth	Description
Auth	POST <code>/auth/register</code>	Non	Création de compte
Auth	POST <code>/auth/login</code>	Non	Connexion et obtention JWT
Auth	GET <code>/me</code>	Oui	Profil utilisateur
Market	GET <code>/search?query=...</code>	Non	Recherche coin (autocomplete)
Market	GET <code>/history/{coin_id}?days=...</code>	Non	Historique prix
Market	GET <code>/predict/{coin_id}?horizon=...</code>	Non	Prévision
Wallet	GET <code>/wallet</code>	Oui	Cash + positions + PnL
Wallet	POST <code>/trade/buy</code>	Oui	Achat crypto
Wallet	POST <code>/trade/sell</code>	Oui	Vente crypto
Bank	GET <code>/bank/balance</code>	Oui	Solde USD
Bank	POST <code>/bank/transfer</code>	Oui	Transfert USD entre users
Benef.	GET/POST/DELETE <code>/bank/beneficiaries</code>	Oui	Gestion bénéficiaires
Orders	GET <code>/orders</code> ; POST <code>/orders</code>	Oui	Gestion des ordres

Problèmes rencontrés et correction

Un problème d'authentification a été rencontré : après un login réussi (200 OK), les endpoints protégés retournaient 401 Unauthorized. La cause provenait d'un format incorrect du payload JWT (champ sub). La correction consiste à générer le token avec l'identifiant utilisateur attendu par `create_access_token`.

4. Application Flutter

Organisation

- lib/services/api.dart : client HTTP + gestion des headers Authorization
- lib/services/session_store.dart : stockage token/refresh (SharedPreferences)
- lib/services/watchlist_store.dart : stockage des favoris (SharedPreferences)
- lib/screens : écrans Marché, Banque, Wallet, Ordres, Profil, Recherche

5. Base de données et migrations

Le stockage est assuré par SQLite (wallet.db). Lors de l'ajout de nouvelles tables/colonnes (bénéficiaires, refresh_token, PnL, etc.), des migrations ont été nécessaires (ALTER TABLE / création de tables).

6. Validation, sécurité et gestion de session

Validation côté backend (règles métier)

- Interdiction de transfert vers soi-même
- Montants min/max + format numérique
- Email destinataire doit exister
- Solde suffisant avant débit
- Contrôles côté buy/sell (montant, quantité, positions)

7. Tests

Tests recommandés :

- Backend (pytest) : login, endpoints protégés, transfert (solde insuffisant, email inexistant), buy/sell, orders
- Frontend (Flutter) : tests de widgets (écrans principaux), tests d'intégration sur scénario login → bank → wallet

8. Conclusion et améliorations

Le projet fournit une démonstration cohérente : API documentée, application mobile fluide, et modules fonctionnels (banque + crypto).

Pistes d'amélioration

- Passage à Alembic pour les migrations
- Notifications (alertes prix)
- Historique de performance du portefeuille (courbe valeur totale)
- Pagination et optimisation des appels marché

Annexes — Captures d'écran

Les figures ci-dessous servent de preuves de fonctionnement (API + application).

A1 — Démarrage du serveur Uvicorn

```
(.venv) PS C:\Users\travail\Documents\L3_informatique\Programation_Mobile\mobileapp-2026-main\server> uvicorn main:app --  
-reload --host 127.0.0.1 --port 8000  
INFO: Will watch for changes in these directories: ['C:\\Users\\travail\\Documents\\L3_informatique\\Programation_Mo  
bile\\mobileapp-2026-main\\server']  
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)  
INFO: Started reloader process [7832] using StatReload  
INFO: Started server process [16236]  
INFO: Waiting for application startup.  
INFO: Application startup complete.
```

A1 — Démarrage du serveur Uvicorn

A2 — Swagger : liste des endpoints

Wallet D mo API 1.1.0 OAS 3.1
[/openapi.json](#)

[Authorize](#)

default ^

GET	/health	Health	^
GET	/users/search	Users Search	^
GET	/prices	Prices	^
GET	/prices/market	Prices Market	^
GET	/price/{coin_id}	Price One	^
POST	/auth/register	Register	^
POST	/auth/login	Login	^
POST	/auth/refresh	Refresh Token	^
GET	/me	Me	^
POST	/auth/change-password	Change Password	^
GET	/search	Search	^

A2 — Swagger : liste des endpoints

A3 — Swagger : wallet / bank / orders

GET	/search Search	✓
GET	/history/{coin_id} History	✓
GET	/predict/{coin_id} Predict	✓
GET	/orders List Orders	✓
POST	/orders Create Order	✓
POST	/orders/{order_id}/cancel Cancel Order	✓
GET	/wallet Wallet	✓
POST	/trade/buy Buy	✓
POST	/trade/sell Sell	✓
GET	/transactions Transactions	✓
GET	/bank/balance Bank Balance	✓
POST	/bank/transfer Bank Transfer	✓
GET	/bank/transactions Bank Transactions	✓
GET	/bank/beneficiaries Bank Beneficiaries List	✓
POST	/bank/beneficiaries Bank Beneficiaries Add	✓
DELETE	/bank/beneficiaries/{id} Bank Beneficiaries Delete	✓

A3 — Swagger : wallet / bank / orders

Analyse



Aperçu

Prévision

Graphique

BITCOIN

Prix : \$68154.80

Horizon : J+7

Zoom : 90 pts

Projection : \$72713.39

Modèle : Ridge(lags=12)

+\$4558.59 (+6.69%)

Actif

bitcoin



Jours d'historique

180



Prévision

J+7



Tip: coinId doit correspondre à CoinGecko.

A5 — Application : Marché (Prévision)



Analyse



Aperçu

Prévision

Graphique

Prévision • BITCOIN

Prix actuel : \$68154.80
Projection (J+7) : \$72713.39
Modèle : Ridge(lags=12)

J+1	\$68751.57
J+2	\$69237.61
J+3	\$70032.79
J+4	\$70861.83
J+5	\$71483.30
J+6	\$72211.81
J+7	\$72713.39

A6 — Application : Marché (Graphique)

Analyse



Aperçu

Prévision

Graphique

Graphique

Historique + Prévision (USD)



● Historique ● Prévision

A7 — Application : Banque (solde + historique)

NeoBank



Banque



Carte virtuelle

\$7450.00

**** * 4821

EXP 08/29 • CVV ***

Bénéficiaires



Aucun bénéficiaire.

Historique



TRANSFER_IN

jojo1@gmail.com • Reçu de jojo1@gmail.com

+\$50.00



INIT

Solde initial (d  mo)

+\$10000.00

A8 — Application : Wallet (positions + PnL + achat)

Portefeuille



Valeur totale

LIVE 10s

10 054,57 \$

Cash USD : 7 450,00 \$

Acheter

Vendre

Positions

BITCOIN

2 094,32 \$

Qty: 0.030739 • PRU: 68 316,92 \$ •
Prix: 68 132,00 \$

-5.68 (-0.27%)

ETHEREUM

510,26 \$

Qty: 0.238376 • PRU: 2 097,53 \$ •
Prix: 2 140,56 \$

+10.26 (+2.05%)

ACHETER



Crypto (coin_id)

bitcoin



A8 — Application : Wallet (positions + PnL + achat)

A9 — Application : Ordres (création LIMIT)



Ordres



Créer un ordre



Side

BUY



Type

LIMIT



+ Créer

Ordres ouverts

Aucun ordre ouvert.



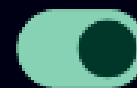
Jojo Jo

jojo@gmail.com

+596696111111

Mode biométrie (option)

Démo: option stockée en local.



Changer le mot de passe



Ancien mot de passe



Nouveau mot de passe



Confirmer nouveau mot de passe

 **Mettre à jour**

A11 — Logs serveur : /wallet 200 OK + /trade/buy 200 OK

```
INFO: 127.0.0.1:62657 - "GET /wallet HTTP/1.1" 200 OK
INFO: 127.0.0.1:62675 - "GET /wallet HTTP/1.1" 200 OK
INFO: 127.0.0.1:62682 - "GET /wallet HTTP/1.1" 200 OK
INFO: 127.0.0.1:57047 - "GET /wallet HTTP/1.1" 200 OK
INFO: 127.0.0.1:57054 - "GET /wallet HTTP/1.1" 200 OK
INFO: 127.0.0.1:59089 - "GET /wallet HTTP/1.1" 200 OK
INFO: 127.0.0.1:59091 - "POST /trade/buy HTTP/1.1" 200 OK
```

A11 — Logs serveur : /wallet 200 OK + /trade/buy 200 OK